

Summary

Permutations and combinations are two problem domains that many programmers don't know how to even begin to approach. While recursion is often an overly complicated solution for common programming problems, it's well suited for the complexity of the tasks in this chapter.

The chapter began with a brief introduction to set theory. This lays the basis for the data structures that our recursive algorithms operate on. A set is a collection of distinct elements. A subset consists of none, some, or all the elements of a set. While sets have no ordering for their elements, a permutation is a specific ordering of the elements in a set. And a combination, which has no ordering, is a specific selection of none, some, or all the elements in a set. A k -combination of a set is a subset of k elements selected from the set.

Permutations and combinations can include an element once or can repeat elements. We call these permutations or combinations without repetition and with repetition, respectively. These are implemented by different algorithms.

This chapter also tackled the balanced parentheses problem that is commonly used in coding interviews. Our algorithm builds the strings of balanced parentheses by starting with a blank string and adding opening and closing parentheses. This approach involves backtracking to earlier strings, making recursion an ideal technique.

Finally, this chapter featured a recursive function for generating power sets—that is, sets of all possible k -combinations of the elements of a set. The recursive function we create to do this is much more efficient than repeatedly calling our combinations function for each possible size of subset.